

with the OS and server. But both of them have the end goal of high quality software releases. They need to rely on each other when some changes take place during the development cycle—and when it is the time to scale up. All developers want scalability, whereas operations teams are focused on application management and its efficiency. Containerisation keeps the separation between the two by isolating their processes and hence helping them to achieve their respective goals in tandem.

Different types of containerisation

Let's now classify the containers based on how they can be used for different applications.

Operating system containers: Operating system containers are actually the virtual environments that share the kernel of the host operating system but, at the same time, provide user space isolation. We can think of OS containers as VMs. We can install and then configure them, and execute different applications, libraries, etc, in the same way that we would on any OS. Just as in the case of a VM, anything running inside a container can only see the different resources that have been assigned to that container. These containers are useful when we want to run a number of the same or different flavours of distros. Often, containers are created using templates or images, which actually determine the structure as well as contents of that container. Hence, we can create containers which have identical environments using the same package versions and configurations across all containers. Different container technologies such as OpenVZ, LXC, Linux VServer and Solaris zones are suitable for creating OS containers.

Application containers: We know that OS containers are designed in such a way that they can run multiple services and processes, but application containers are designed keeping in mind that they will package and run a single service. So, in spite of the fact that they share the same kernel of the host, there are many differences between these two types of containers. Whenever any application container is launched, it runs a single process that actually runs our application when we create different containers per application. This is very different from the OS containers, where we have multiple services running on the same operating system.

Different container technologies like Rocket and Docker are examples of application containers. Any RUN commands we specify in the Dockerfile are to create a new layer for the container. In the end, Docker combines such layers and runs our containers. Layering helps Docker to increase the reuse and reduce duplication. This is really helpful when we want to create different containers for our components. We can start with a base image, which is common for all components and then we just add different layers that are specific to our component. It also helps when we want to roll back our changes, as we can simply switch to the old layers. There is no overhead involved in doing so.

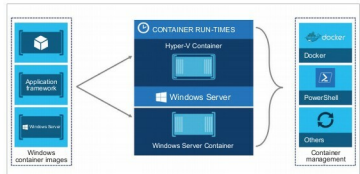


Figure 3: How Windows Server Container works and its management (Image source: [googleimages.com](https://www.google.com/search?q=googleimages.com))

How secure is containerisation?

It is a common belief that containers are less secure than virtual machines, because if there is any vulnerability in the container host kernel then it could provide a way into those containers that share it. This is even true for a hypervisor, but since a hypervisor provides less functionality than a Linux kernel (as it implements file systems, application process controls, networking, etc) it presents a much smaller attack surface. There is one benefit to containerisation over virtualisation — we need to secure the host OS and every other OS running on top of it in case of virtualisation, but with a containerisation model, we just need to secure that one host OS.

For the past few years, a great deal of effort has been devoted to developing software to enhance the security of containers. For instance, Docker (and other such container systems) now include a signing infrastructure, which allows administrators to sign the container images in order to prevent different untrusted containers from being deployed. Given below are some of the ways in which containerisation security is being taken care of.

1. It is not always the case that a signed and trusted container is secure to run, as vulnerabilities may also be discovered in some of the software in the container even after it has been signed. That's why Docker and other such containers offer different container security scanning solutions, which can notify administrators if any of the container images have vulnerabilities that could be exploited.
2. There is also some specialised container security software that has been developed in order to keep a strict vigilance on different security aspects. For instance, Twistlock-It offers software which profiles a container's expected behaviour, networking activities (like source and destination IP addresses and ports) and also certain storage practices so that any unexpected or malicious behaviour can be flagged.
3. Another such container security company is Polyverse. It takes a slightly different approach. It makes use of the fact that different containers can be started even in a fraction of a second, to relaunch the containerised applications in a good state every few seconds so that